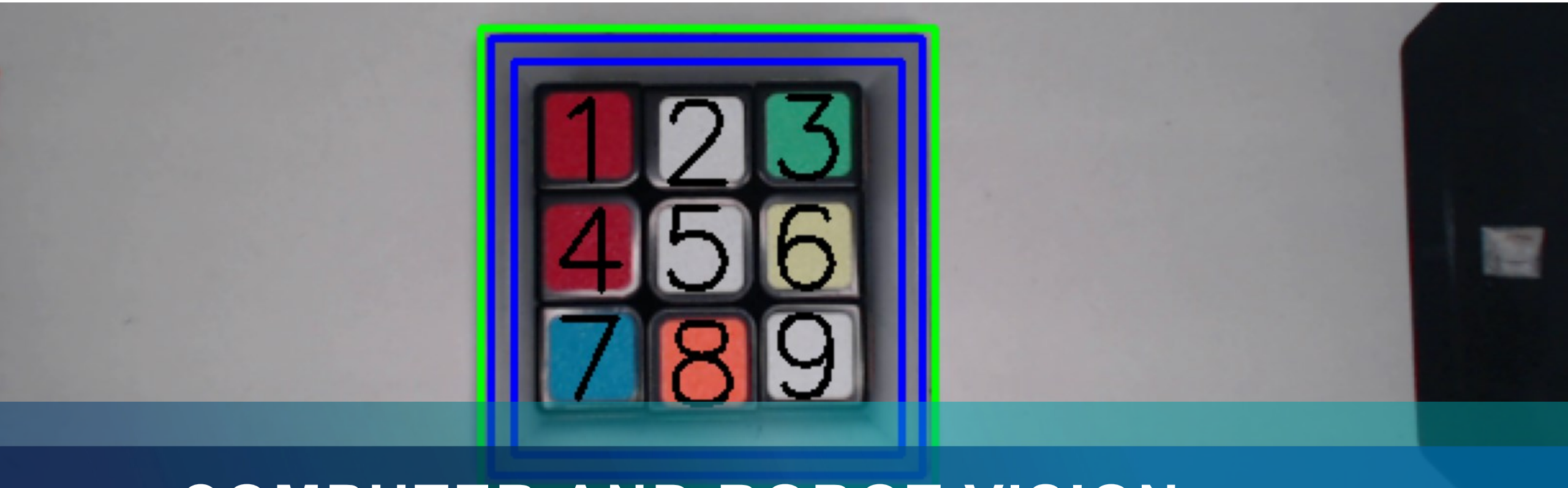
Campus Sontheim

HOCHSCHULE HEILBRONN



› COMPUTER AND ROBOT VISION

Rubik's Cube color detection

Gerstlauer, Lukas; Söns, Tim

Computer and Robot Vision | Automotive Systems Engineering | WS 24/25

HOCHSCHULE HEILBRONN

AGENDA

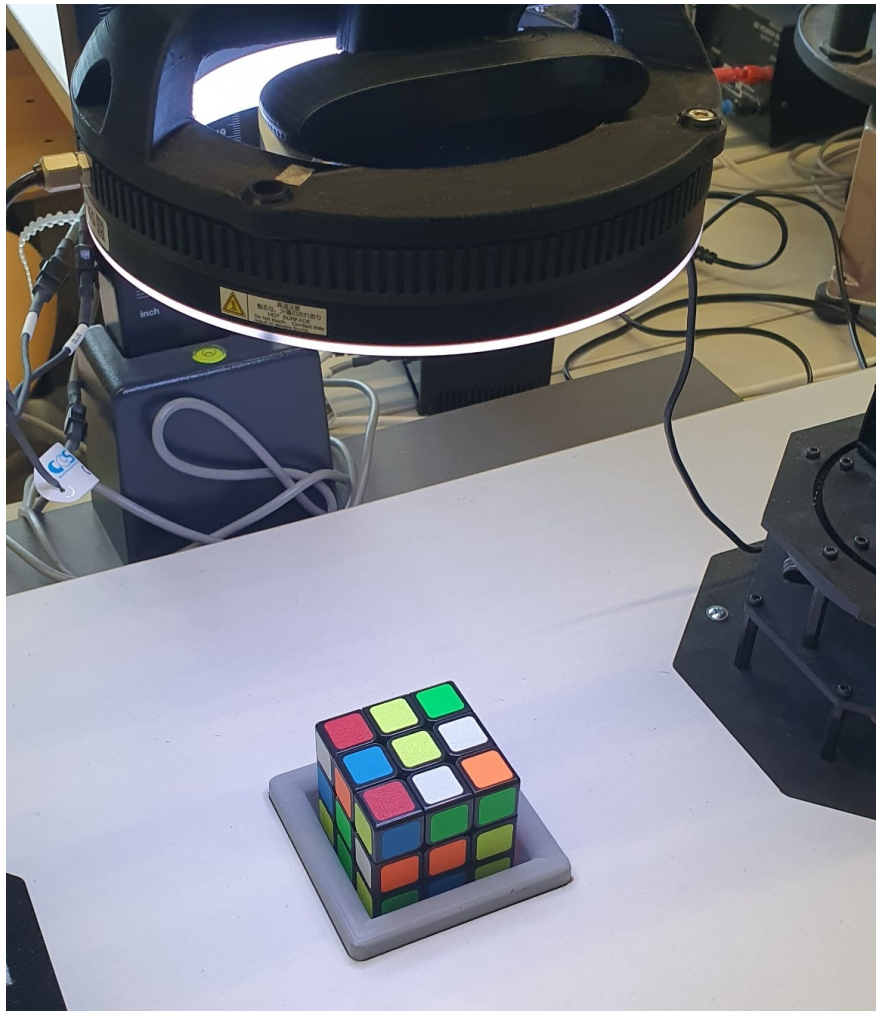
- Project description
- Challenges of the Project
- Process description
- Demo-picture
- Demo-live

PROJECT DESCRIPTION

The aim is to recognize the 9 coloured squares of a Rubik's cube under different lighting conditions in the lab environment

Experimental setup:

1. Rubiks cube in grey socket
2. LED-Lightsource
3. Webcam



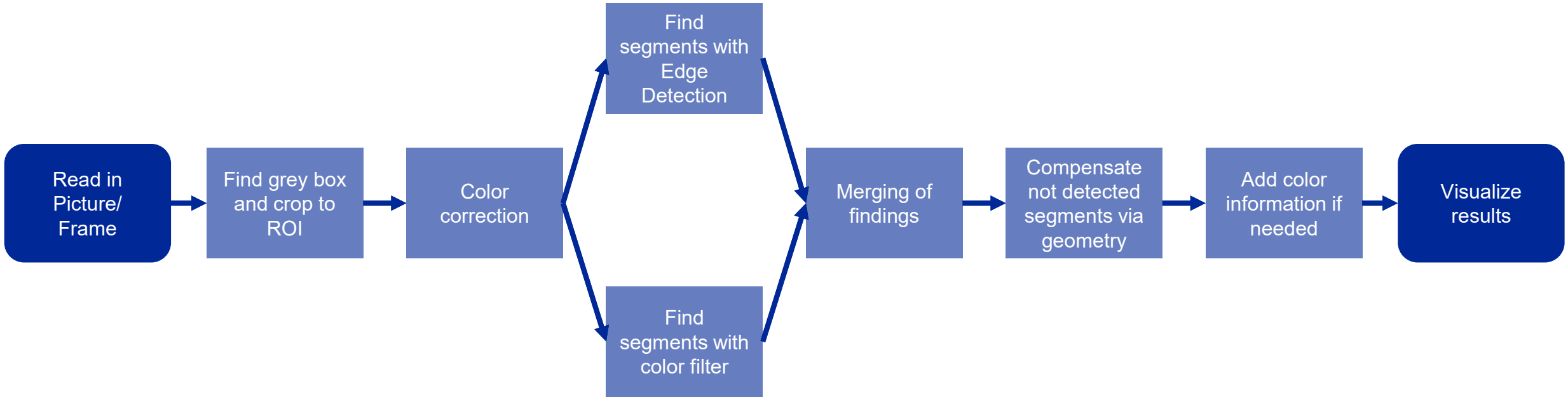
HOCHSCHULE HEILBRONN

CHALLANGES

- Due to the smooth surface of the cube there a many reflections
- The webcam isn‘t mounted in the center of the LED circle which interfers with the exposure
- Color correction is limited to small changes in the lighting conditions

HOCHSCHULE HEILBRONN

PROCESS DESCRIPTION - FLOW CHART



PROCESS DESCRIPTION – GREY FRAME DETECTION

1. Smoothing with bilateral filter
2. Edge detection with Canny
3. Closing of edges
4. Search for largest square contour
5. Reduce size of outer contour by 5%
6. Get inner contour by reducing size of outer contour by 10%

HOCHSCHULE HEILBRONN

PROCESS DESCRIPTION – COLOR CORRECTION

Calibration to get color conditions of Ground Truth:

Ground Truth: Color values of test image, color thresholds based on that

1. Get RGB-values of grey frame
2. Calculate correction values c_0, c_R, c_G, c_B
3. Recalibrate image

$$\hat{c}_0 = \frac{1}{3}\left(\frac{\sigma'_r}{\sigma_r} + \frac{\sigma'_g}{\sigma_g} + \frac{\sigma'_b}{\sigma_b}\right)$$

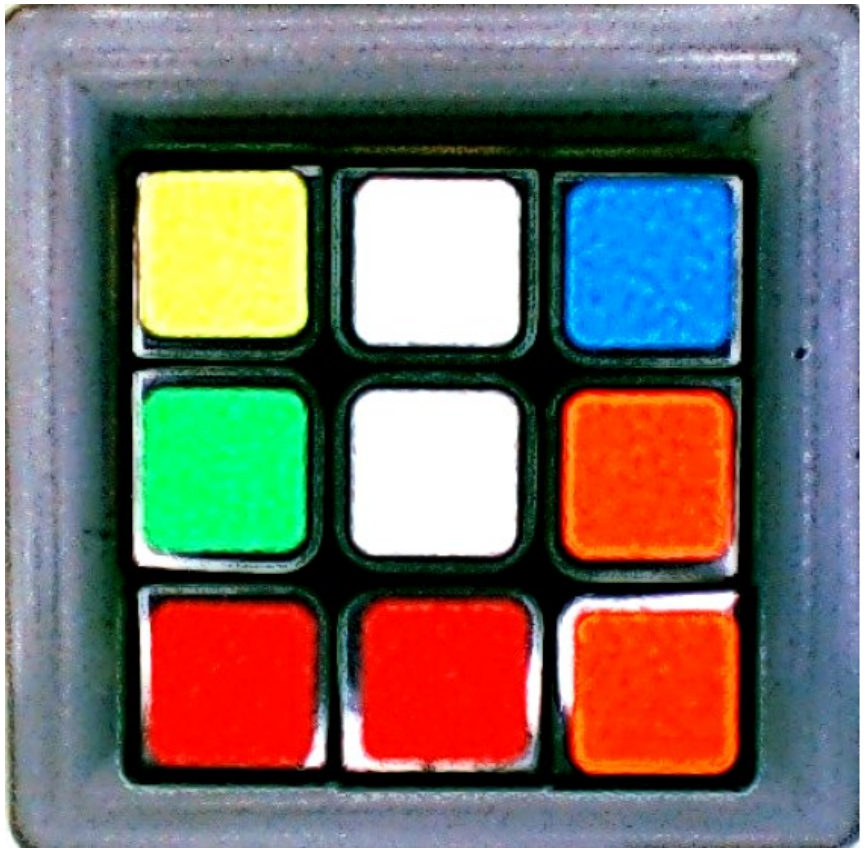
$$\hat{c}_r = \frac{\langle r' \rangle}{\hat{c}_0} - \langle r \rangle; \hat{c}_g = \frac{\langle g' \rangle}{\hat{c}_0} - \langle g \rangle; \hat{c}_b = \frac{\langle b' \rangle}{\hat{c}_0} - \langle b \rangle$$

$$s''(x,y) = \frac{s'(x,y)}{\hat{c}_0} - \hat{c}_i : \; i = r, g, b$$

PROCESS DESCRIPTION – COLOR CORRECTION

Before correction

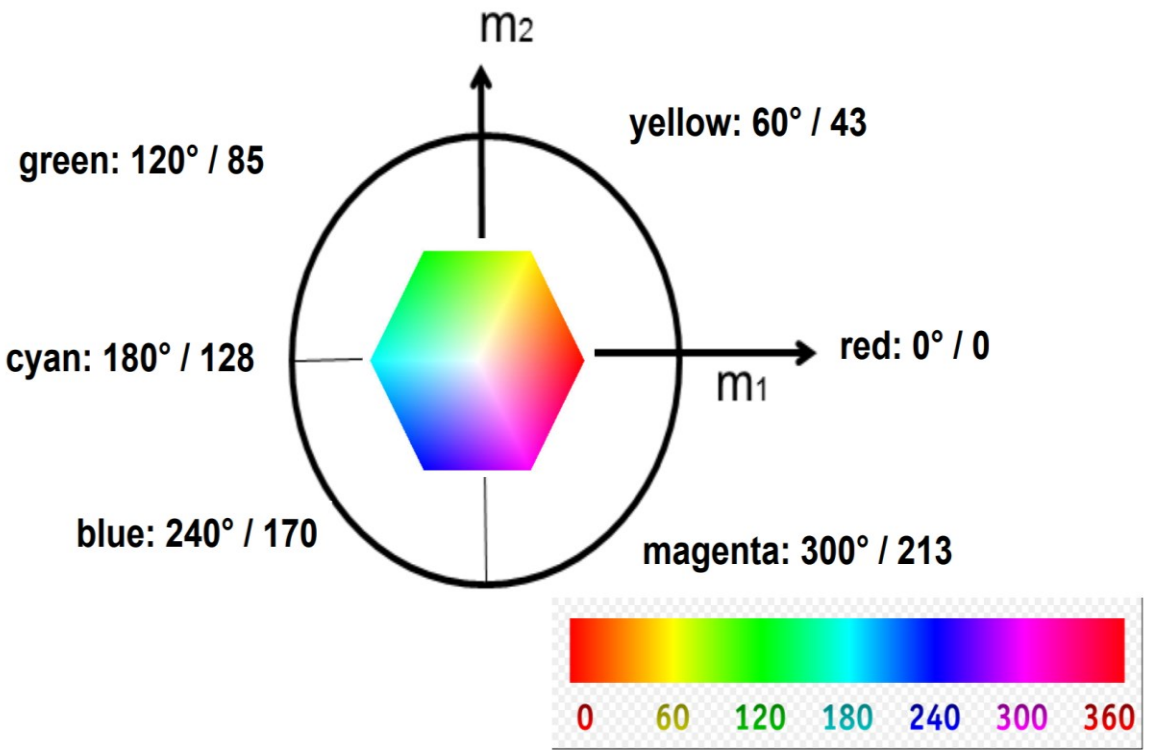
After correction



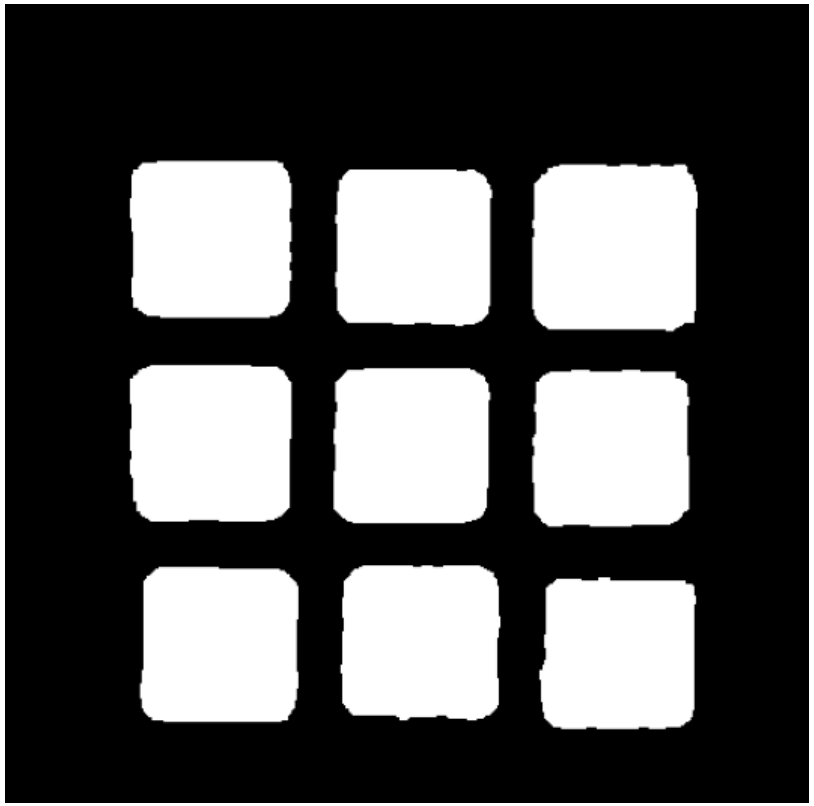
HOCHSCHULE HEILBRONN

PROCESS DESCRIPTION – COLOR FILTERING

1. Transform image in HSV color space
2. Filter image for specified color thesholds
3. Get contours of object with specified color values
4. Filter contours based on size, filling factor, squareness
5. Get attributes for every found object



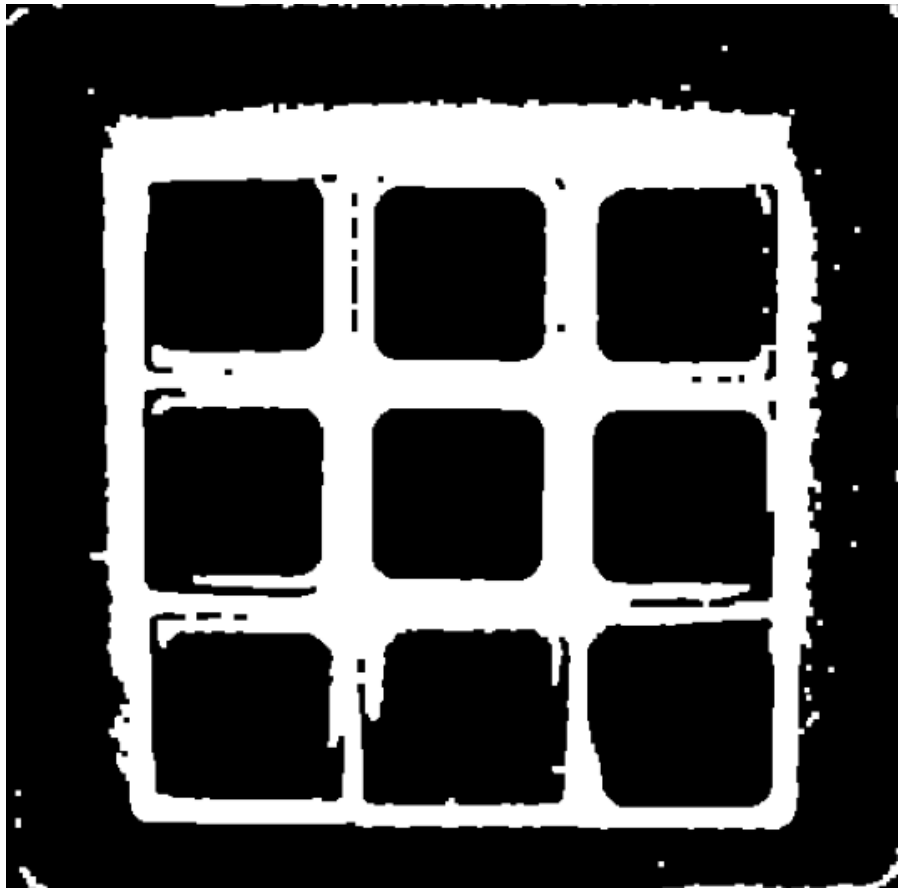
PROCESS DESCRIPTION – COLOR FILTERING



Mask after color filtering and contour filtering

PROCESS DESCRIPTION – EDGE FILTERING

1. Get mask of black pixels
2. Find contours in mask image
3. Filter contours by area
4. Calculate focus point of contour



Mask after edge filtering without contour filtering

PROCESS DESCRIPTION – GRID POSITION

1. Split x-,y-coordinate of focus points
2. Calculate a distance threshold besed on mean height/width of Boundingboxes
3. Group coordinates in groups for rows and colums
4. Make sure for three elements in each group

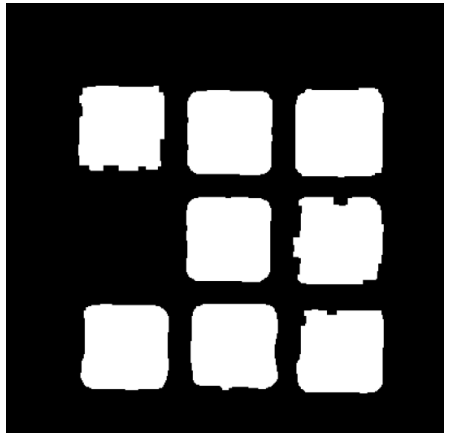
Necessary condition:
One element in each row and each colum

Numbered grid with all fields

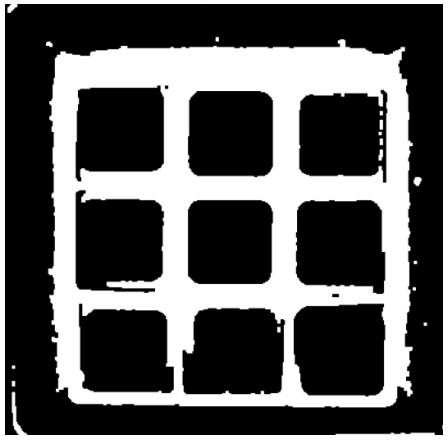
PROCESS DESCRIPTION – MERGING OF RESULTS

Iterate through the grid positions and look for entries in dictionaries:

1. If available in both dicts: Merge with a factorized mean
2. If grid position only available in one dict: Take that entry
3. If grid position isn‘t available in any dict: compensate through geometry



Mask after color filtering with one missing object

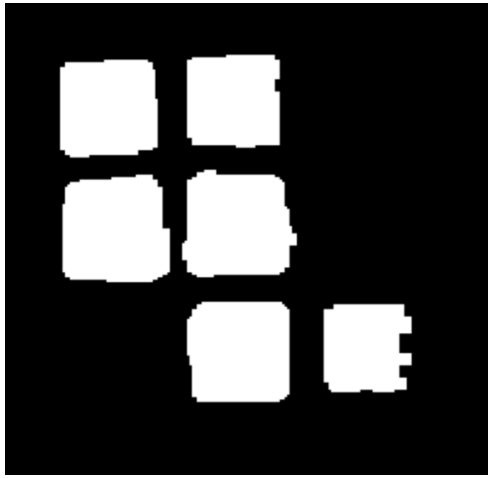


Mask after edge filtering detection with every object

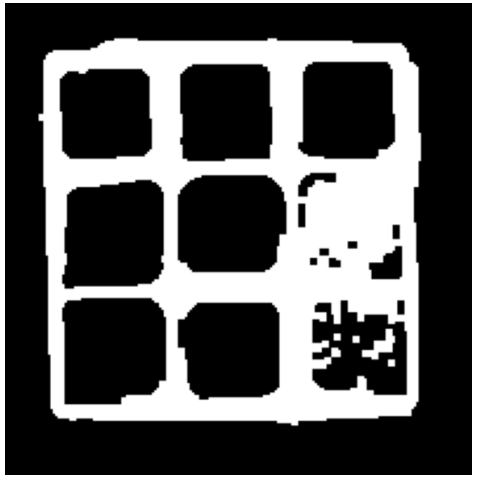
PROCESS DESCRIPTION – COMPENSATION

Mean x and y groups with three elements each

Look for missing grid position and create dict entry based on centre points



Mask after color filtering with missing objects



Mask after edge filtering detection with missing object

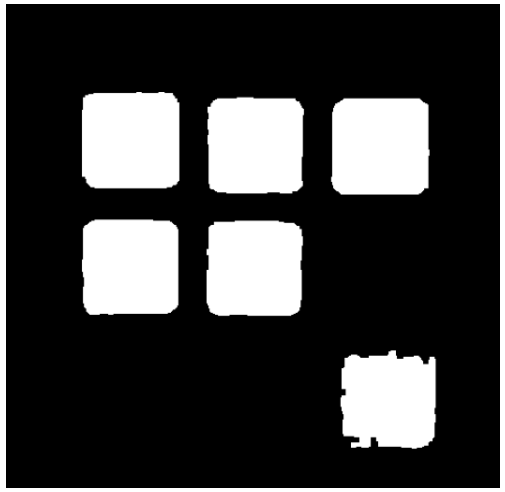
Final image with all objects (compensated sixth tile)

PROCESS DESCRIPTION – BACKWARDS COLOR DET.

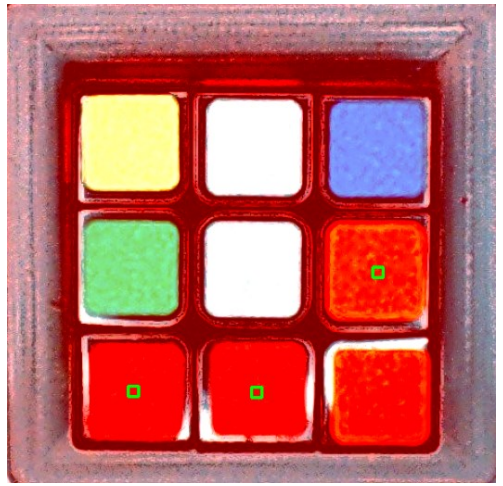
Color detection if object detected by Edge detection or Compensation, not by color filtering

1. Get color values in a rectangle around the focus point of found objects

2. Compare to extended specified color thresholds, no more filtering by size, filling factor,…



Mask after color filtering with missing objects



Small ROIs in the middle of the tiles to detect the color

PROCESS DESCRIPTION – RESULT / OUTPUT

Output: Merged dictionary with nine entrys with six keys each

- Bounding Box
- Focus point
- Color
- Grid position
- Average hue value
- Detected way

Position	Color	Focus Point	Detected
1	Rot	(41, 43)	edge
2	Rot	(84, 42)	edge
3	None	(127, 41)	edge
4	Blau	(44, 87)	edge
5	Blau	(85, 82)	edge
6	Rot	(127, 84)	interpolated
7	Rot	(43, 127)	edge
8	Grun	(84, 127)	edge
9	Rot	(127, 127)	interpolated

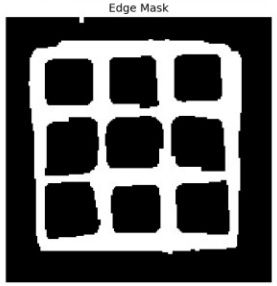
LIMITS OF THE SYSTEM

- Does not work when the cube is rotated in the image
- Orange and red color areas are very close to each other, risk of confusion in the border area
- Color correction reaches its limits with excessive color deviations
- Detection of the grey frame not always exactly at its edge (shadows, angle to the camera) → inaccurate color correction

DEMO VIA RECORDED PICTURES

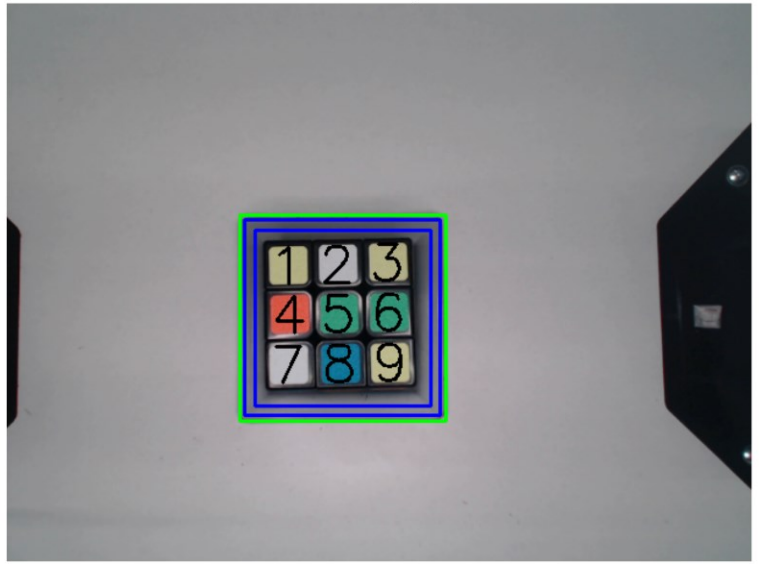
Corrected Image

Edge Mask



Colored Mask

Processed Image



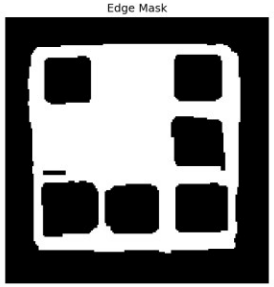
Position	Color	Focus Point	Detected
1	Gelb	[41, 43]	both
2	Weiss	[84, 42]	both
3	Gelb	[127, 40]	both
4	Rot	[43, 85]	both
5	Grun	[85, 84]	both
6	Grun	[128, 83]	both
7	Weiss	[42, 128]	both
8	Blau	[86, 127]	both
9	Gelb	[130, 126]	both

HOCHSCHULE HEILBRONN

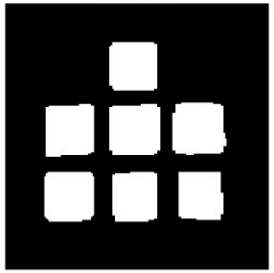
DEMO VIA RECORDED PICTURES

Corrected Image

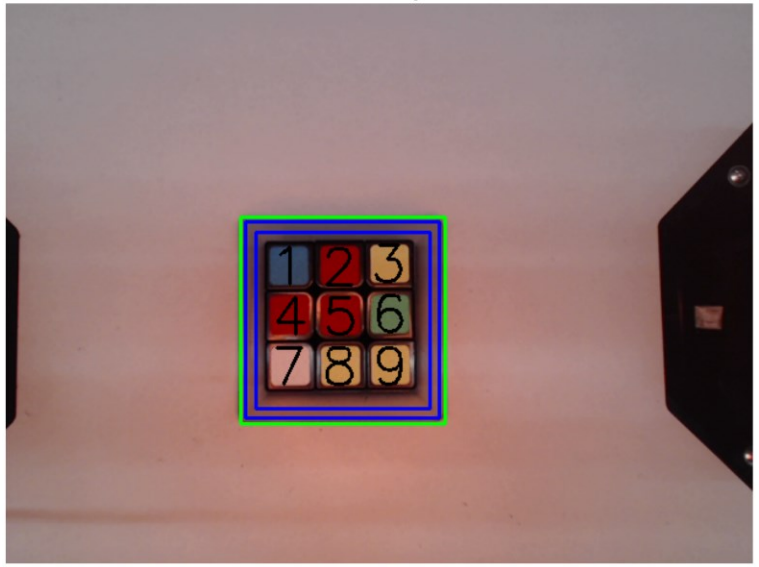
Edge Mask



Colored Mask



Processed Image

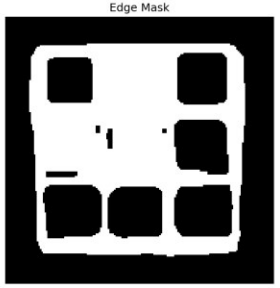


Position	Color	Focus Point	Detected
1	Blau	[41, 41]	edge
2	Rot	[85, 42]	color
3	Gelb	[127, 39]	edge
4	Rot	[43, 84]	color
5	Rot	[86, 84]	color
6	Grun	[128, 82]	both
7	Weiss	[42, 127]	both
8	Gelb	[85, 127]	both
9	Gelb	[129, 127]	both

DEMO VIA RECORDED PICTURES

Corrected Image

Edge Mask



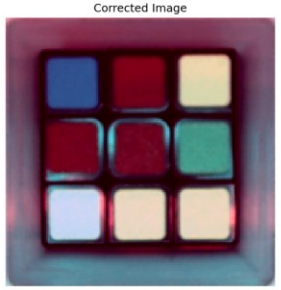
Colored Mask

Processed Image

Position	Color	Focus Point	Detected
1	Blau	[43, 42]	color
2	Rot	[87, 41]	color
3	Gelb	[129, 40]	both
4	Rot	[44, 85]	color
5	Rot	[88, 86]	color
6	Grun	[129, 84]	both
7	None	[44, 127]	edge
8	Gelb	[86, 128]	both
9	Gelb	[130, 127]	both

DEMO VIA RECORDED PICTURES

Corrected Image



Edge Mask

Colored Mask

Processed Image

Position	Color	Focus Point	Detected
1	Blau	[43, 43]	both
2	Rot	[88, 43]	color
3	Gelb	[130, 41]	edge
4	Rot	[44, 86]	color
5	Rot	[89, 87]	color
6	Grun	[130, 86]	both
7	Weiss	[45, 128]	edge
8	Weiss	[88, 129]	both
9	Weiss	[132, 128]	edge

THANKS FOR YOUR ATTENTION